Лабораторна робота № 1

Вхідний аналіз набору даних

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи · **Максимальна оцінка:** 4 бали

Мета роботи

Пригадати роботу в Jupyter Notebook, бібліотеці `pandas` та побудову графіків — і зробити це не на навчальному прикладі, а на реальному файлі, який ніхто не готував спеціально для вас.

Ця робота виконується **до першої лекції**. Нічого нового знати не потрібно: усе, що знадобиться, ви вже проходили в попередніх курсах. Усі поняття, яких ви тут торкнетеся мимохідь, отримають назви й пояснення на лекціях 3 і 4.

Дані

Файл `20171129_FINAL_FINAL.xlsx`, аркуш `Sheet1` — спостереження космічної погоди за 28 серпня — 22 вересня 2017 року, зібрані з трьох різних джерел.

Файл видається **як є**, рівно в тому вигляді, у якому його колись вивантажили. Ніхто не приводив його до ладу, і саме в цьому сенс: справжні дані здебільшого надходять саме такими.

Що ви маєте зробити

Етап	Зміст	Де
1	Розібратися, що взагалі лежить у файлі	аудиторно
2	Виділити три таблиці й привести їх до робочого вигляду	аудиторно
3	Скласти паспорт даних: що є, чого бракує, що виглядає дивно	аудиторно
4	Об'єднати три таблиці в одну	аудиторно
5	Побудувати п'ять графіків, кожен — відповідь на питання	аудиторно + самостійно
6	Порівняти два графіки одних і тих самих даних	самостійно
7	Записати три спостереження про дані	самостійно
8	Декларація про використання інструментів ШІ	самостійно

Що здається

1. Цей файл `.ipynb` із заповненими розв'язками. **Ноутбук має виконуватися згори донизу на чистому ядрі** (Kernel → Restart & Run All) без помилок — це перевіряється першою дією на захисті.
2. Файл `master_hourly.csv` — об'єднана таблиця з етапу 4.

3. Експорт ноутбука у HTML або PDF.
4. Заповнена декларація (етап 8). Без неї фінальна самоперевірка не проходить.

Критерії оцінювання

Бали	Що досягнуто
0	Дані не опрацьовано, графіки не побудовано або побудовано з грубими помилками
1	Таблиці прочитано й кілька графіків побудовано, але типи графіків не відповідають даним, дивних зна
2	Таблиці розібрано й об'єднано, графіки коректні та підписані, але висновків під ними немає або вони
4	Усе вище виконано, знайдено й пояснено дивні значення, під кожним графіком є змістовний висновок

Головне про оцінку: бали ставляться не за наявність графіка, а за речення під ним. Код, який нічого не пояснює, коштує двох балів з чотирьох.

Нагадування про Jupyter

Якщо давно не працювали — три речі, які знадобляться сьогодні:

- `Shift+Enter` виконує клітинку, `Esc` `A` / `Esc` `B` додає клітинку вище / нижче, `Esc` `M` перетворює клітинку на текстову (Markdown), `Esc` `Y` — назад на код.
- **Порядок виконання має значення.** Якщо ви правили клітинки в довільному порядку, результат може залежати від історії запусків. Перед здачею обов'язково **Kernel → Restart & Run All** — так ви побачите ноутбук очима викладача.
- Текстові клітинки — не прикраса. Висновки пишуться в них, а не в коментарях до коду.

Крок 0. Ваші дані та варіант

Заповніть три змінні й виконайте обидві клітинки. Друга надрукує **ваше індивідуальне завдання** — воно відрізняється від завдання сусіда.

```
ПІБ = 'Недосіка Назарій Вадимович'        # напр. 'Шевченко Тарас'
ГРУПА = 'ШІ-32'      # напр. 'КН-31'
ВАРІАНТ = 14     # ваш номер за списком групи

assert ПІБ and ГРУПА and ВАРІАНТ, 'Заповніть ПІБ, ГРУПУ та ВАРІАНТ'
```

```
# --- клітинку не редагувати: вона видає ваше персональне завдання ---------
ПОТОКИ = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0']
```

```
ПАРИ = [('швидкість вітру', 'густина вітру'),
        ('швидкість вітру', 'потік P>10'),
        ('густина вітру', 'потік P>1'),
        ('радіопотік F10.7', 'потік P>10'),
        ('радіопотік F10.7', 'швидкість вітру'),
        ('потік E>0.8', 'потік P>10')]

АГРЕГАТИ = ['середнє', 'максимум', 'медіана', 'мінімум']

ПИТАННЯ = [
    'Скільки годин за все вікно ваш потік перевищував своє власне медіанне з
    'У яку добу середнє значення вашого потоку було найбільшим? А найменшим?
    'О котрій годині доби потік E>0.8 у середньому найбільший? Побудуйте гра
    'Яка частка рядків об’єднаної таблиці має пропуск хоча б в одному стовпц
    'У який день радіопотік F10.7 був найбільшим і чи збігається цей день з
    'Порівняйте медіану вашого потоку за перший і за другий тиждень спостере
    'Скільки годин поспіль тривала найдовша неперервна серія «активних» годи
    'Який зі стовпців об’єднаної таблиці має найбільший розкид відносно влас
    'Яку частку сумарного значення вашого потоку за все вікно дали 24 години
    'Скільки різних діб потрапило у групу «активні» і скільки — у групу «дуж
]

i = int(ВАРІАНТ)
завдання = {
    'ваш потік':                    ПОТОКИ[(i * 3 + 1) % 8],
    'ваша пара для графіка 3':      ' та '.join(ПАРИ[(i * 5 + 2) % 6]),
    'ваш агрегат для етапу 4':      АГРЕГАТИ[(i * 5 + 1) % 4],
    'ваше додаткове питання':       ПИТАННЯ[(i * 11 + 5) % 10],
}

print('ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · %s, %s, варіант %d' % (ПІБ, ГРУПА, i))
print('=' * 78)
for k, v in завдання.items():
    print('%-28s %s' % (k + ':', v))
```

```
ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · Недосіка Назарій Вадимович, ШІ-32, варіант 14
==============================================================================
ваш потік:                  P>30
ваша пара для графіка 3:    швидкість вітру та густина вітру
ваш агрегат для етапу 4:    мінімум
ваше додаткове питання:     Скільки різних діб потрапило у групу «активні» і
```

Далі за текстом **«ваш потік»** означає стовпець, названий у першому рядку завдання.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option('display.width', 200)
pd.set_option('display.max_columns', 40)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                     'axes.spines.right': False, 'font.size': 9})
```

```
ФАЙЛ = '20171129_FINAL_FINAL.xlsx'
```

Етап 1. Що взагалі лежить у файлі

Перше правило роботи з незнайомими даними: спершу подивитися, потім рахувати.

Завдання 1.1

Спробуйте прочитати аркуш `Sheet1` звичайним способом — `pd.read_excel(ФАЙЛ)` — і виведіть перші рядки та типи стовпців.

Очікуваний результат: результат буде дивним. Подивіться на назви стовпців і на те, які там типи даних. Поясніть у наступній клітинці, чому так вийшло.

```
# ВАШ КОД
raw_default = pd.read_excel(ФАЙЛ, sheet_name='Sheet1')
display(raw_default.head())
print(raw_default.dtypes)
```

	Unnamed: 0	ftp://ftp.swpc.noaa.gov/pub/lists/particle/	Unnamed: 2	Unnamed: 3
0	NaN	NaN	NaN	NaN
1	NaN	:Data_list: 20170901_Gs_part_5m.txt	NaN	NaN
2	NaN	:Created: 2017 Sep 02 0011 UTC	NaN	NaN
3	NaN	# Prepared by the U.S. Dept. of Commerce, NOAA...	NaN	NaN
4	NaN	# Please send comments and suggestions to SWPC...	NaN	NaN

```
Unnamed: 0                                          float64
ftp://ftp.swpc.noaa.gov/pub/lists/particle/          object
Unnamed: 2                                           object
Unnamed: 3                                           object
Unnamed: 4                                           object
Unnamed: 5                                           object
Unnamed: 6                                           object
Unnamed: 7                                           object
Unnamed: 8                                           object
Unnamed: 9                                           object
Unnamed: 10                                          object
Unnamed: 11                                          object
Unnamed: 12                                          object
Unnamed: 13                                          object
Unnamed: 14                                          object
Unnamed: 15                                         float64
Unnamed: 16                                          object
Unnamed: 17                                          object
Unnamed: 18                                          object
Unnamed: 19                                          object
Unnamed: 20                                         float64
Unnamed: 21                                         float64
Unnamed: 22                                         float64
Unnamed: 23                                         float64
Unnamed: 24                                         float64
Unnamed: 25                                         float64
http://umtof.umd.edu/pm/crn/                         object
Unnamed: 27                                          object
Unnamed: 28                                          object
Unnamed: 29                                          object
Unnamed: 30                                          object
Unnamed: 31                                          object
dtype: object
```

Відповідь 1.1: *(чому звичайне читання дало саме такий результат)*

Тому що функція сприйняла перший рядок тексту за заголовки, а мікс тексту з числами получили тип object

Завдання 1.2

Прочитайте аркуш ще раз, але **без заголовка**: `pd.read_excel(ФАЙЛ, header=None)`. Виведіть його розмір і подивіться на перші 30 рядків.

```
# ВАШ КОД
RAW = pd.read_excel(ФАЙЛ, header=None)
print("Розмір таблиці:", RAW.shape)
display(RAW.head(30))
```

```
Розмір таблиці: (7226, 32)
```

	0	1	2	3	4	5	
0	NaN	ftp://ftp.swpc.noaa.gov/pub/lists/particle/	NaN	NaN	NaN	NaN	Na
1	NaN	NaN	NaN	NaN	NaN	NaN	Na
2	NaN	:Data_list: 20170901_Gs_part_5m.txt	NaN	NaN	NaN	NaN	Na
3	NaN	:Created: 2017 Sep 02 0011 UTC	NaN	NaN	NaN	NaN	Na
4	NaN	# Prepared by the U.S. Dept. of Commerce, NOAA...	NaN	NaN	NaN	NaN	Na
5	NaN	# Please send comments and suggestions to SWPC...	NaN	NaN	NaN	NaN	Na
6	NaN	# Na...	NaN	NaN	NaN	NaN	Na
7	NaN	# Label: P > 1 = Particles at >1 Mev	NaN	NaN	NaN	NaN	Na
8	NaN	# Label: P > 5 = Particles at >5 Mev	NaN	NaN	NaN	NaN	Na
10	NaN	# Label: P >30 = Particles at >30 Mev	NaN	NaN	NaN	NaN	Na
11	NaN	# Label: P >50 = Particles at >50 Mev	NaN	NaN	NaN	NaN	Na
12	NaN	# Label: P>100 = Particles at >100 Mev	NaN	NaN	NaN	NaN	Na
13	NaN	# Label: E>0.8 = Electrons at >0.8 Mev	NaN	NaN	NaN	NaN	Na
14	NaN	# Label: E>2.0 = Electrons at >2.0 Mev	NaN	NaN	NaN	NaN	Na
15	NaN	# Label: E>4.0 = Electrons at >4.0 Mev	NaN	NaN	NaN	NaN	Na
16	NaN	# Units: Particles = Protons/cm2-s-sr	NaN	NaN	NaN	NaN	Na
17	NaN	# Units: Electrons = Electrons/cm2-s-sr	NaN	NaN	NaN	NaN	Na
18	NaN	# Source: GOES-15	NaN	NaN	NaN	NaN	Na
19	NaN	# Location: W135	NaN	NaN	NaN	NaN	Na
20	NaN	# Missing data: -1.00e+05	NaN	NaN	NaN	NaN	Na
21	NaN	NaN	NaN	NaN	NaN	NaN	Na
22	NaN	5-minute GOES-15 Solar Particle and Electron ...	NaN	NaN	NaN	NaN	Na
23	NaN	NaN	NaN	NaN	NaN	Modified	Second
24	NaN	# UT	C D	ate	Time	Julian	of th
25	NaN	# YR	MO	DA	HHMM	Day	Da

Завдання 1.3

Для кожного стовпця порахуйте, скільки в ньому заповнених клітинок (`RAW.notna().sum()`), і виведіть результат.

*Очікуваний результат: 32 числа. Серед них будуть нулі та згустки великих значень. За цими згустками видно, що в аркуші **не одна таблиця, а кілька**, вставлені поруч.*

Питання: скільки таблиць в аркуші і які стовпці займає кожна?

```
print(RAW.notna().sum())
```

```
0         0
1      7223
2      7202
3      7202
4      7202
5      7203
6      7203
7      7198
8      7198
9      7198
10     7198
11     7198
12     7198
13     7198
14     7198
15     7198
16     7198
17     7198
18     7198
19     7198
20     7198
21     7198
22     7198
23     7198
24     7198
25     7198
26     7198
27     7198
28     7198
29     7198
30     7198
31     7198
dtype: int64
```

26	NaN	2017	8	28	0	57993	
27	NaN	2017	8	28	5	57993	3(
28	NaN	2017	8	28	10	57993	6(
29	NaN	2017	8	28	15	57993	9(

Відповідь 1.3: *(скільки таблиць і які стовпці кожна займає)*

Всього 3 таблиці A - 1 - 14 стовпці B - 16 - 19 стовпці C - 26 - 31 стовпці

Завдання 1.4

Над кожною таблицею є кілька текстових рядків — це службовий опис від тих, хто дані збирав. Виведіть їх і випишіть для кожної таблиці **джерело даних та одиниці вимірювання**.

Це найкорисніші п'ять хвилин роботи: жодної з цих відомостей у самій таблиці немає, і без них ви не знатимете, у чому вимірюєте.

```
# ВАШ КОД
for col in RAW.columns:
    text_data = RAW[col].iloc[:25].dropna().astype(str)
    print(f"Стовпець {col}")
    for val in text_data:
        print(val)
    print()
```

```
Стовпець 0

Стовпець 1
ftp://ftp.swpc.noaa.gov/pub/lists/particle/
:Data_list: 20170901_Gs_part_5m.txt
:Created: 2017 Sep 02 0011 UTC
# Prepared by the U.S. Dept. of Commerce, NOAA, Space Weather Prediction Cer
# Please send comments and suggestions to SWPC.Webmaster@noaa.gov
#
# Label: P > 1 = Particles at >1 Mev
# Label: P > 5 = Particles at >5 Mev
# Label: P >10 = Particles at >10 Mev
# Label: P >30 = Particles at >30 Mev
# Label: P >50 = Particles at >50 Mev
# Label: P>100 = Particles at >100 Mev
# Label: E>0.8 = Electrons at >0.8 Mev
# Label: E>2.0 = Electrons at >2.0 Mev
# Label: E>4.0 = Electrons at >4.0 Mev
# Units: Particles = Protons/cm2-s-sr
# Units: Electrons = Electrons/cm2-s-sr
# Source: GOES-15
# Location: W135
# Missing data: -1.00e+05
5-minute  GOES-15 Solar Particle and Electron Flux
# UT

Стовпець 2
C D

Стовпець 3
ate
```

```
Стовпець 4
Time

Стовпець 5
Modified
Julian

Стовпець 6
Seconds
of the

Стовпець 7

Стовпець 8

Стовпець 9

Стовпець 10

Стовпець 11

Стовпець 12

Стовпець 13
```

Відповідь 1.4:

Таблиця	Джерело	Одиниці вимірювання
A	GOES-15	Protons/cm2-s-sr, Electrons/cm2-s-sr
B	Daily Solar Data	Not specified
C	SOHO CELIAS Proton Monitor	Proton speed [km/sec]

Етап 2. Три таблиці в робочому вигляді

У файлі три таблиці. Ось що в них лежить — решту ви вже знайшли самі:

Таблиця	Стовпці аркуша	Що це	Один рядо
A	B–O	вимірювання із супутника GOES-15: потоки частинок різної енергії	5 хвилин
B	Q–T	добовий показник сонячної активності F10.7	доба
C	Y–AF	швидкість і густина сонячного вітру	година

Завдання 2.1

Виділіть кожну таблицю в окремий `DataFrame` — назвіть їх `A`, `B` і `C`. Дайте стовпцям осмислені назви й перетворіть значення на числа (`pd.to_numeric(..., errors='coerce')` стане в пригоді).

```
# ВАШ КОД
A = RAW.iloc[25:, 1:15].copy()
B = RAW.iloc[25:, 16:20].copy()
```

```
C = RAW.iloc[25:, 24:32].copy()

A.columns = ['YR', 'MO', 'DA', 'HHMM', 'MJD', 'Secs', 'P>1', 'P>5', 'P>10',
B.columns = ['YR', 'MO', 'DA', 'F10.7']
C.columns = ['YR', 'MO', 'DA', 'HHMM', 'MJD', 'Secs', 'швидкість вітру', 'гу

A = A.apply(pd.to_numeric, errors='coerce').dropna(how='all')
B = B.apply(pd.to_numeric, errors='coerce').dropna(how='all')
C = C.apply(pd.to_numeric, errors='coerce').dropna(how='all')

print("Розмір A:", A.shape)
print("Розмір B:", B.shape)
print("Розмір C:", C.shape)
```

```
Розмір A: (7200, 14)
Розмір B: (25, 4)
Розмір C: (601, 8)
```

Завдання 2.2

У кожній таблиці дата й час записані **по-різному** й розкидані по кількох стовпцях. Зберіть для кожної таблиці один стовпець `ts` типу `datetime`.

Після цього обов'язково виведіть для кожної таблиці найранішу й найпізнішу дату.

Очікуваний результат: усі три таблиці мають лежати в межах серпня–вересня 2017 року. Якщо у вас вийшов інший рік — придивіться, як записано рік у тій таблиці, і виправте.

```
# ВАШ КОД
C = RAW.iloc[27:, 26:32].copy()
C.columns = ['YR', 'MO', 'DA', 'HH', 'швидкість вітру', 'густина вітру']
C = C.apply(pd.to_numeric, errors='coerce').dropna(how='all')

A['ts'] = pd.to_datetime(A['YR'].astype(int).astype(str) + '-' + A['MO'].ast

C['ts'] = pd.to_datetime(C['YR'].astype(int).astype(str) + '-' + C['MO'].ast

B['ts'] = pd.to_datetime(B['YR'].astype(int).astype(str) + '-' + B['MO'].ast

display(C[['YR', 'MO', 'DA', 'HH', 'ts']].head())

print("Таблиця A: від", A['ts'].min(), "до", A['ts'].max())
print("Таблиця B: від", B['ts'].min(), "до", B['ts'].max())
print("Таблиця C: від", C['ts'].min(), "до", C['ts'].max())
```

	YR	MO	DA	HH	ts
27	17.0	8.0	28.0	0.0	2017-08-28 00:00:00
28	17.0	8.0	28.0	1.0	2017-08-28 01:00:00
29	17.0	8.0	28.0	2.0	2017-08-28 02:00:00
30	17.0	8.0	28.0	3.0	2017-08-28 03:00:00
31	17.0	8.0	28.0	4.0	2017-08-28 04:00:00

```
Таблиця A: від 2017-08-28 00:00:00 до 2017-09-21 23:55:00
Таблиця B: від 2017-08-28 00:00:00 до 2017-09-21 00:00:00
Таблиця C: від 2017-08-28 00:00:00 до 2017-09-22 00:00:00
```

Самоперевірка етапу 2. Клітинка має виконатися без помилок.

```
assert len(A) == 7200, 'у таблиці A має бути 7200 рядків, а є %d' % len(A)
assert len(B) == 25,   'у таблиці B має бути 25 рядків, а є %d' % len(B)
assert len(C) == 601,  'у таблиці C має бути 601 рядок, а є %d' % len(C)
for назва, т in [('A', A), ('B', B), ('C', C)]:
    рік = pd.to_datetime(т['ts']).dt.year
    assert рік.min() == 2017 and рік.max() == 2017, 'у таблиці %s рік не 201
print('Етап 2 пройдено')
```

```
Етап 2 пройдено
```

Етап 3. Паспорт даних

Завдання 3.1

Для кожної таблиці виведіть: розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків і описові статистики (`describe()`).

```
# ВАШ КОД
def print_passport(df, name):
    print(f"Паспорт таблиці {name}")
    print(f"Розмір (рядки, стовпці): {df.shape}")
    print(f"Кількість повних дублікатів: {df.duplicated().sum()}\n")

    info_df = pd.DataFrame({
        'Тип даних': df.dtypes,
        'Кількість пропусків': df.isna().sum()
    })
    print("Типи стовпців та пропуски")
    display(info_df)

    print("\nОписові статистики")
    display(df.describe())
    print("\n\n")
```

```
print_passport(A, 'A')
print_passport(B, 'B')
print_passport(C, 'C')
```

```
Паспорт таблиці A
Розмір (рядки, стовпці): (7200, 15)
Кількість повних дублікатів: 0

Типи стовпців та пропуски
```

	Тип даних	Кількість пропусків
YR	float64	0
MO	float64	0
DA	float64	0
HHMM	float64	0
MJD	float64	0
Secs	float64	0
P>1	float64	3
P>5	float64	3
P>10	float64	3
P>30	float64	3
P>50	float64	3
P>100	float64	3
E>0.8	float64	3
E>2.0	float64	3
ts	datetime64[ns]	0

Описові статистики

	YR	MO	DA	HHMM	MJD	Se
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.0000
mean	2017.0	8.840000	13.960000	1177.500000	58005.000000	43050.0000
min	2017.0	8.000000	1.000000	0.000000	57993.000000	0.0000
25%	2017.0	9.000000	7.000000	588.750000	57999.000000	21525.0000
50%	2017.0	9.000000	13.000000	1177.500000	58005.000000	43050.0000
75%	2017.0	9.000000	19.000000	1766.250000	58011.000000	64575.0000

max	2017.0	9.000000	31.000000	2355.000000	58017.000000	86100.0000
std	0.0	0.366632	8.775483	692.481902	7.211603	24943.1134

```
Паспорт таблиці B
Розмір (рядки, стовпці): (25, 5)
Кількість повних дублікатів: 0

Типи стовпців та пропуски
```

	Тип даних	Кількість пропусків
YR	float64	0
MO	float64	0
DA	float64	0
F10.7	float64	0
ts	datetime64[ns]	0

```
Описові статистики
```

	YR	MO	DA	F10.7	ts
count	25.0	25.000000	25.000000	25.000000	25
mean	2017.0	8.840000	13.960000	92.680000	2017-09-09 00:00:00
min	2017.0	8.000000	1.000000	71.000000	2017-08-28 00:00:00
25%	2017.0	9.000000	7.000000	74.000000	2017-09-03 00:00:00
50%	2017.0	9.000000	13.000000	84.000000	2017-09-09 00:00:00
75%	2017.0	9.000000	19.000000	107.000000	2017-09-15 00:00:00
max	2017.0	9.000000	31.000000	140.000000	2017-09-21 00:00:00
std	0.0	0.374166	8.955817	22.206831	NaN

```
Паспорт таблиці C
Розмір (рядки, стовпці): (601, 7)
Кількість повних дублікатів: 0

Типи стовпців та пропуски
```

	Тип даних	Кількість пропусків
YR	float64	0
MO	float64	0
DA	float64	0
HH	float64	0
швидкість вітру	float64	0

густина вітру	float64	0
ts	datetime64[ns]	0

Описові статистики

	YR	MO	DA	HH	швидкість вітру	густина вітру	ts
count	601.0	601.000000	601.000000	601.000000	601.000000	601.000000	601
mean	17.0	8.840266	13.973378	11.480865	506.547421	2.907654	2009-12-00
min	17.0	8.000000	1.000000	0.000000	-1.000000	-1.000000	2008

Завдання 3.2 — дивні значення

Подивіться на рядок `min` у таблиці `describe()` для таблиці C.

Швидкість і густина — фізичні величини, які **не можуть бути від'ємними**.

Знайдіть від'ємні значення, порахуйте, скільки їх, і поверніться до службового опису з завдання 1.4: там написано, що вони означають.

Замініть ці значення на `NaN` і покажіть, як після цього змінилися середнє й мінімум.

Очікуваний результат: два числа «до» і «після» для кожного зі стовпців.

```
# ВАШ КОД
import numpy as np

cols_A = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0']
A[cols_A] = A[cols_A].mask(A[cols_A] < 0, np.nan)

B['F10.7'] = B['F10.7'].mask(B['F10.7'] < 0, np.nan)

cols_C = ['швидкість вітру', 'густина вітру']
C[cols_C] = C[cols_C].mask(C[cols_C] < 0, np.nan)

display(C[['швидкість вітру', 'густина вітру']].describe().loc[['min', 'max'
```

	швидкість вітру	густина вітру
min	277.0	0.1
max	806.0	31.7

Відповідь 3.2: *(що це за значення, скільки їх, як вони спотворювали середнє)*

Це аномальні відємні значення. Зявляються коли датчик ламається, або втрачає звязок. Через це і псується середнє значення

Завдання 3.3 — питання на подумати

У таблиці A є стовпці, для яких `describe()` слухняно рахує середнє й стандартне відхилення, хоча ці числа не мають жодного сенсу.

Знайдіть їх і поясніть своїми словами, чому середнє для них безглузде.

Підказка: не кожне число в таблиці є результатом вимірювання.

> Це питання не має однієї «правильної» відповіді з підручника — на нього ви відповідаєте з власного здорового глузду. Формальне пояснення буде на лекції 3.

```
# ВАШ КОД
display(A[['YR', 'MO', 'DA', 'HHMM', 'MJD', 'Secs']].describe())
```

	YR	MO	DA	HHMM	MJD	Secs
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.00000
mean	2017.0	8.840000	13.960000	1177.500000	58005.000000	43050.00000
std	0.0	0.366632	8.775483	692.481902	7.211603	24943.11349
min	2017.0	8.000000	1.000000	0.000000	57993.000000	0.00000
25%	2017.0	9.000000	7.000000	588.750000	57999.000000	21525.00000
50%	2017.0	9.000000	13.000000	1177.500000	58005.000000	43050.00000
75%	2017.0	9.000000	19.000000	1766.250000	58011.000000	64575.00000
max	2017.0	9.000000	31.000000	2355.000000	58017.000000	86100.00000

Відповідь 3.3: *(які це стовпці та чому середнє для них не має сенсу)*

Це стовпці часу, для часу нема сенсу рахувати середнє

Етап 4. Об'єднання трьох таблиць в одну

Таблиці мають різний крок у часі: 5 хвилин, година і доба. Щоб працювати з ними разом, потрібна спільна сітка. Беремо **годину**.

Кожна таблиця потребує своєї дії:

1. **A: 5 хвилин → година.** Кілька рядків згортаються в один — це `resample` + агрегація.
2. **C: уже година.** Просте з'єднання по спільному стовпцю `ts` — `merge`.
3. **B: доба → година.** Навпаки, одне значення розтягується на 24 години доби.

Завдання 4.1

Зведіть таблицю A до погодинної сітки. Для кожного потоку порахуйте **два** числа за годину: середнє і максимум.

```
# ВАШ КОД
A_hourly = A.set_index('ts').resample('H').agg(['mean', 'max'])

A_hourly.columns = [f"{col}_{stat}" for col, stat in A_hourly.columns]
A_hourly = A_hourly.reset_index()

display(A_hourly.head())
```

```
C:\Users\nazus\AppData\Local\Temp\ipykernel_3380\1918681547.py:2: FutureWarni
  A_hourly = A.set_index('ts').resample('H').agg(['mean', 'max'])
```

	ts	YR_mean	YR_max	MO_mean	MO_max	DA_mean	DA_max	HHMM_mean	HHMM
0	2017-08-28 00:00:00	2017.0	2017.0	8.0	8.0	28.0	28.0	27.5	
1	2017-08-28 01:00:00	2017.0	2017.0	8.0	8.0	28.0	28.0	127.5	
2	2017-08-28 02:00:00	2017.0	2017.0	8.0	8.0	28.0	28.0	227.5	
3	2017-08-28 03:00:00	2017.0	2017.0	8.0	8.0	28.0	28.0	327.5	
4	2017-08-28 04:00:00	2017.0	2017.0	8.0	8.0	28.0	28.0	427.5	

Завдання 4.2

У вашому завданні названо агрегат (`середнє`, `максимум`, `медіана` або `мінімум`).

Візьміть ваш потік за одну добу, порахуйте для нього по годинах ваш агрегат і ще один на ваш вибір, і побудуйте обидві криві на одному графіку.

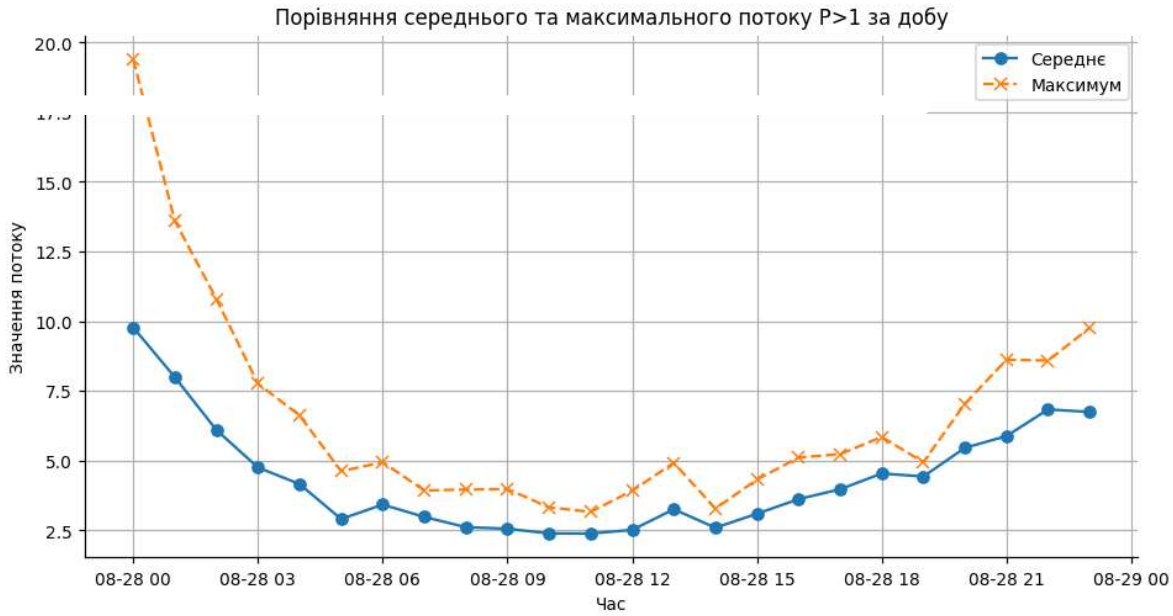
Питання: чим вони відрізняються і в якій ситуації різниця між ними стане суттєвою?

```
# ВАШ КОД
import matplotlib.pyplot as plt
```

```
sample_day = A_hourly.head(24)

plt.figure(figsize=(10, 5))
plt.plot(sample_day['ts'], sample_day['P>1_mean'], label='Середнє', marker='
plt.plot(sample_day['ts'], sample_day['P>1_max'], label='Максимум', marker='

plt.title('Порівняння середнього та максимального потоку P>1 за добу')
plt.xlabel('Час')
plt.ylabel('Значення потоку')
plt.legend()
plt.grid(True)
plt.show()
```



Відповідь 4.2: *(чим відрізняються криві та коли це важливо)*

Середнє показує загальну тенденцію, максимум - піки. Це важливо під час спалахів, максимум стрімко летить вгору, а середнє згладжує різкий зріст.

Завдання 4.3

Об'єднайте три таблиці в одну — назвіть її `M`. Виведіть її розмір і всі рядки, у яких є хоча б один пропуск.

Зверніть увагу: таблиці закінчуються не в один і той самий момент. Тому результат залежить від того, який тип об'єднання ви оберете (`how='inner'` чи `how='outer'`). Спробуйте обидва, подивіться, чим відрізняється кількість рядків, і поясніть свій вибір.

```
# ВАШ КОД

C_hourly = C[['ts', 'швидкість вітру', 'густина вітру']].copy()
B['date'] = B['ts'].dt.date
A_h = A.set_index('ts').resample('H').mean(numeric_only=True).reset_index()

M = pd.merge(A_h, C_hourly, on='ts', how='outer')
M['date'] = M['ts'].dt.date
M = pd.merge(M, B[['date', 'F10.7']], on='date', how='left').drop(columns=['

print("Розмір M:", M.shape)
display(M[M.isna().any(axis=1)].head())
```

```
Розмір M: (601, 18)
C:\Users\nazus\AppData\Local\Temp\ipykernel_3380\1239280907.py:5: FutureWarni
  A_h = A.set_index('ts').resample('H').mean(numeric_only=True).reset_index()
```

	ts	YR	MO	DA	HHMM	MJD	Secs	P>1	P>5	
175	2017-09-04 07:00:00	2017.0	9.0	4.0	727.5	58000.0	26850.0	6.465833	0.262083	0.17
176	2017-09-04 08:00:00	2017.0	9.0	4.0	827.5	58000.0	30450.0	4.010833	0.256000	0.19
177	2017-09-04 09:00:00	2017.0	9.0	4.0	927.5	58000.0	34050.0	3.268333	0.303417	0.22
178	2017-09-04 10:00:00	2017.0	9.0	4.0	1027.5	58000.0	37650.0	3.179167	0.252167	0.18
179	2017-09-04 11:00:00	2017.0	9.0	4.0	1127.5	58000.0	41250.0	5.274167	0.295083	0.18

Відповідь 4.3: *(який тип об'єднання обрано, скільки рядків дає кожен і чому)*

Обрано how='outer' оскільки таблиці покривають різні часові діапазони. outer зберігає всі записи додаючи NaN там, де дані відсутні

Завдання 4.4 — поділ на групи

Щоб далі було що з чим порівнювати, поділіть години на три групи за значенням **максимуму потоку P>10 за годину**:

Група	Умова
тиха	менше 10
активна	від 10 до 1000

Група	Умова
`дуже активна`	1000 і більше

Додайте до `M` стовпець `група` і виведіть, скільки годин потрапило в кожну.

> Це не вигадані числа: саме такі пороги використовує метеослужба космічної погоди NOAA. Чому вони саме такі — питання не сьогоднішньої роботи.

```
# ВАШ КОД

import numpy as np

conditions = [
    (M['P>10'] < 10),
    (M['P>10'] >= 10) & (M['P>10'] < 1000),
    (M['P>10'] >= 1000)
]
choices = ['тиха', 'активна', 'дуже активна']

M['група'] = np.select(conditions, choices, default='невідомо')

print("Кількість годин за групами активності:")
display(M['група'].value_counts())
```

```
Кількість годин за групами активності:
група
тиха            416
активна         167
дуже активна     17
невідомо          1
Name: count, dtype: int64
```

Самоперевірка етапу 4.

```
assert M is not None, 'таблиця M не побудована'
assert 'ts' in M.columns, 'у таблиці M потрібен стовпець ts'
assert 'група' in M.columns, 'у таблиці M потрібен стовпець «група»'
assert 590 <= len(M) <= 605, 'очікується близько 600 рядків, а є %d — переві
print('Етап 4 пройдено:', len(M), 'рядків')
```

```
Етап 4 пройдено: 601 рядків
```

Етап 5. П'ять графіків — п'ять питань

Графік будується як **відповідь на питання**, а не «щоб був».

Вимоги до кожного графіка:

- підписані обидві осі, з одиницями вимірювання;
- заголовок;
- під графіком, у текстовій клітинці, — **одне речення про дані**. Не «на графіку зображено потік протонів», а твердження на кшталт «більшість годин потік не перевищує 60, але кілька годин дають значення у сотні разів більші».

> Не кожне питання має відповідь «так, зв'язок є». Якщо на графіку нічого не видно — так і напишіть. Відсутність зв'язку — це теж результат, і чесно про неї написати важливіше, ніж підбирати ракурс, у якому «щось наче проглядається».

Графік 1. Як розподілені значення вашого потоку?

Побудуйте гістограму вашого потоку (середнє за годину). Порахуйте окремо середнє й медіану цього стовпця.

Якщо гістограма вийде нечитабельною — стовпчик біля нуля і порожньо далі — не залишайте так. Спробуйте побудувати гістограму від логарифма значень (`np.log10`) і подивіться, що зміниться.

Питання: чому середнє й медіана так сильно відрізняються?

```
# ВАШ КОД
import matplotlib.pyplot as plt
import numpy as np

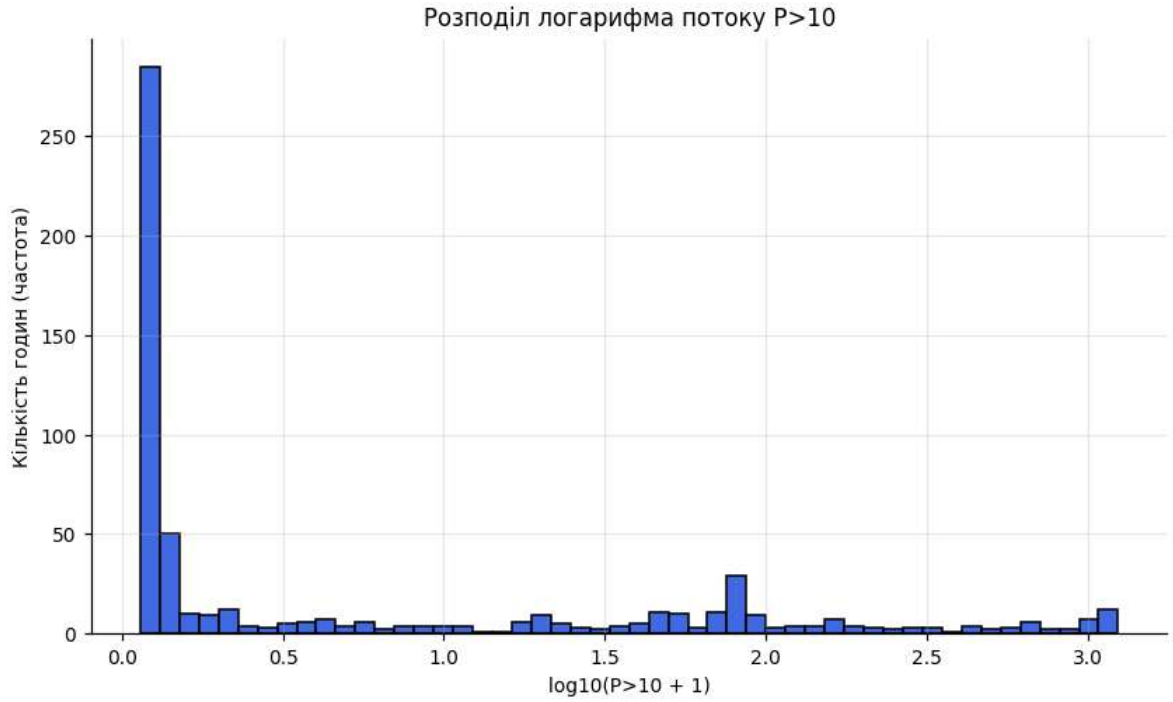
mean_val = M['P>10'].mean()
median_val = M['P>10'].median()
print(f"Середнє: {mean_val:.2f} | Медіана: {median_val:.2f}")

plt.figure(figsize=(9, 5))

log_data = np.log10(M['P>10'].dropna() + 1)

plt.hist(log_data, bins=50, color='royalblue', edgecolor='black')
plt.title('Розподіл логарифма потоку P>10')
plt.xlabel('log10(P>10 + 1)')
plt.ylabel('Кількість годин (частота)')
plt.grid(True, alpha=0.3)
plt.show()
```

```
Середнє: 76.45 | Медіана: 0.34
```



Висновок: Більшість годин рівень потоку є низьким і стабільним, але окремі годинні інтервали демонструють колосальні викиди у сотні разів вищі за норму, що створює сильну асиметрію в розподілі.

Середнє і медіана відрізняються через те, що середнє враховує абсолютно всі числа, а медіана до них стійка

Графік 2. Чи відрізняється швидкість вітру між групами годин?

Побудуйте діаграму розмаху (`boxplot`) швидкості сонячного вітру окремо для кожної групи з завдання 4.4.

```
# ВАШ КОД
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(9, 5))
sns.boxplot(data=M, x='група', y='швидкість вітру', palette='Set2', order=['

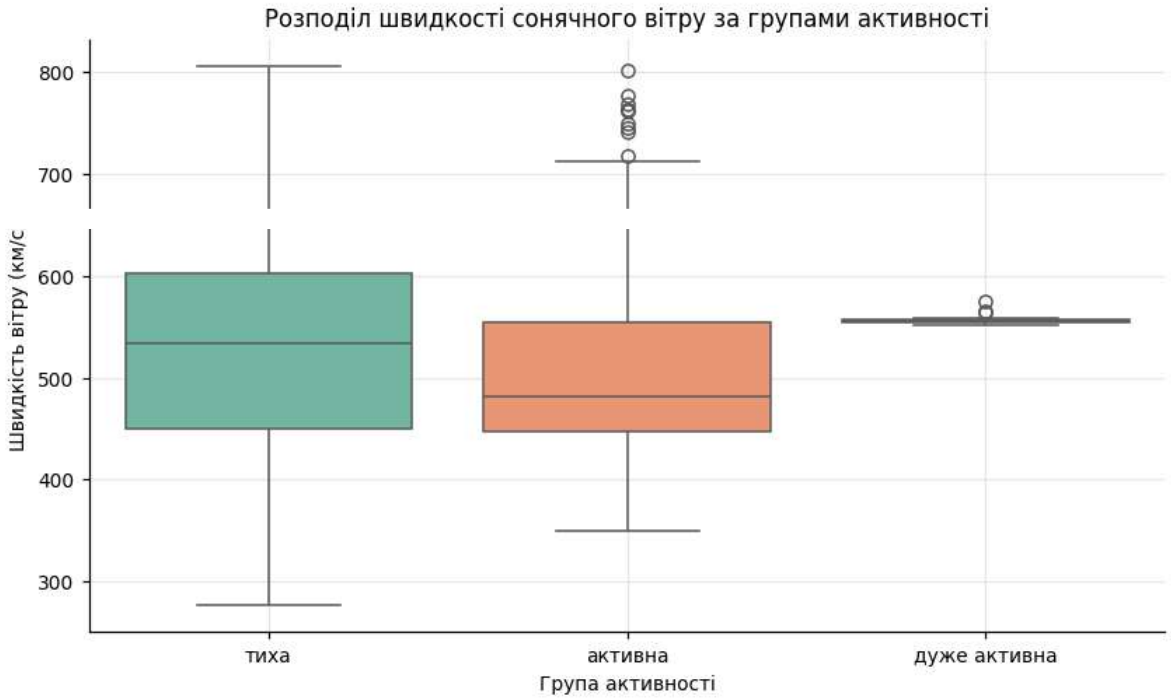
plt.title('Розподіл швидкості сонячного вітру за групами активності')
plt.xlabel('Група активності')
plt.ylabel('Швидкість вітру (км/с)')
```

```
plt.grid(True, alpha=0.3)
plt.show()
```

```
C:\Users\nazus\AppData\Local\Temp\ipykernel_3380\1019080253.py:6: FutureWarni

Passing `palette` without assigning `hue` is deprecated and will be removed i

  sns.boxplot(data=M, x='група', y='швидкість вітру', palette='Set2', order=[
```



Висновок: Зі зростанням рівня активності сонячних потоків медіанна швидкість сонячного вітру помітно збільшується, а діапазон її коливань розширюється

Графік 3. Чи пов'язані змінні вашої пари?

Побудуйте діаграму розсіювання для пари з вашого завдання. Порахуйте коефіцієнт кореляції між ними.

```
# ВАШ КОД
import matplotlib.pyplot as plt

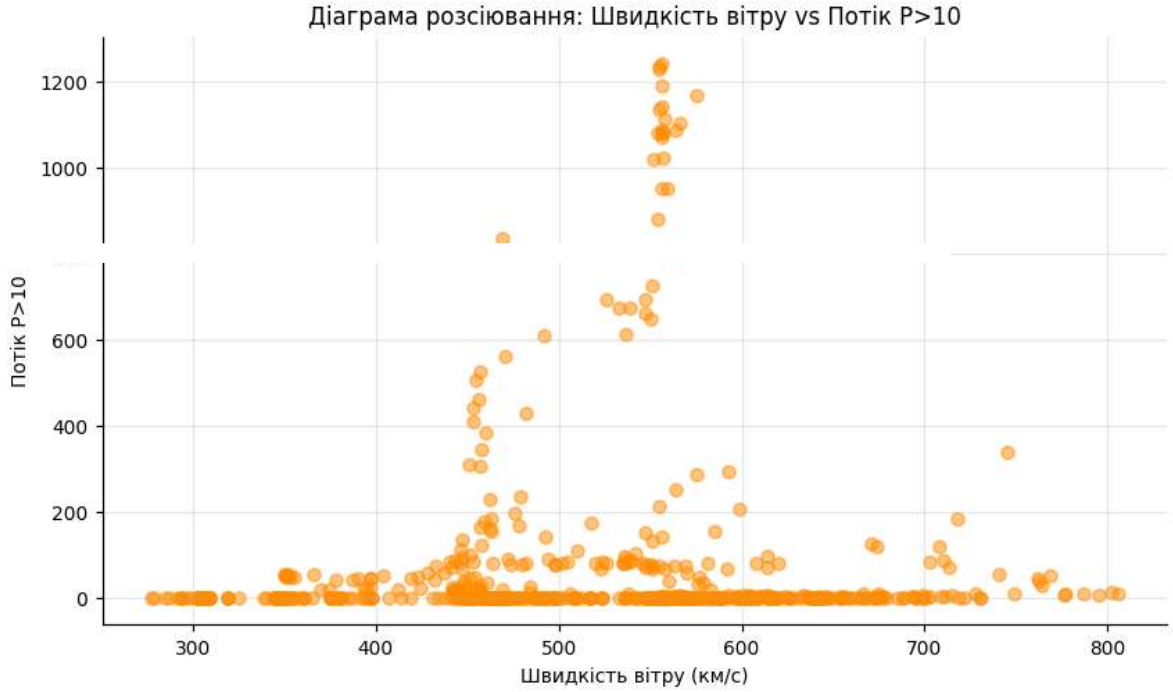
clean_data = M[['швидкість вітру', 'P>10']].dropna()

corr = clean_data['швидкість вітру'].corr(clean_data['P>10'])
print(f"Коефіцієнт кореляції: {corr:.3f}")
```

```
plt.figure(figsize=(9, 5))
plt.scatter(clean_data['швидкість вітру'], clean_data['P>10'], alpha=0.5, co

plt.title('Діаграма розсіювання: Швидкість вітру vs Потік P>10')
plt.xlabel('Швидкість вітру (км/с)')
plt.ylabel('Потік P>10')
plt.grid(True, alpha=0.3)
plt.show()
```

```
Коефіцієнт кореляції: 0.071
```



Висновок: Між швидкістю сонячного вітру та потоком протонів спостерігається прямий помірний зв'язок: зі збільшенням швидкості вітру значення потоку також мають тенденцію до зростання.

Графік 4. Скільки годин у кожній групі?

Побудуйте стовпчикову діаграму кількості годин за групами. Підпишіть кожен стовпчик кількістю годин і відсотком від загальної кількості.

```
# ВАШ КОД
import matplotlib.pyplot as plt

counts = M['група'].value_counts()
total = len(M)
```

```
plt.figure(figsize=(9, 5))
bars = plt.bar(counts.index, counts.values, color=['skyblue', 'orange', 'cri

plt.title('Кількість годин в кожній групі активності')
plt.xlabel('Група активності')
plt.ylabel('Кількість годин')
plt.grid(True, axis='y', alpha=0.3)

for bar in bars:
    yval = bar.get_height()
    percent = (yval / total) * 100
    plt.text(bar.get_x() + bar.get_width()/2.0, yval + 20, f'{yval} г.\n({pe

plt.ylim(0, max(counts.values) * 1.15)
plt.show()
```



Висновок: Більшість часу космічна погода у тихому стані, а активні та дуже активні періоди становлять лише маленьку частину загального часу спостережень

Графік 5. Як усе це виглядало в часі?

Побудуйте лінійний графік максимуму потоку P>10 за годину за весь період спостережень.

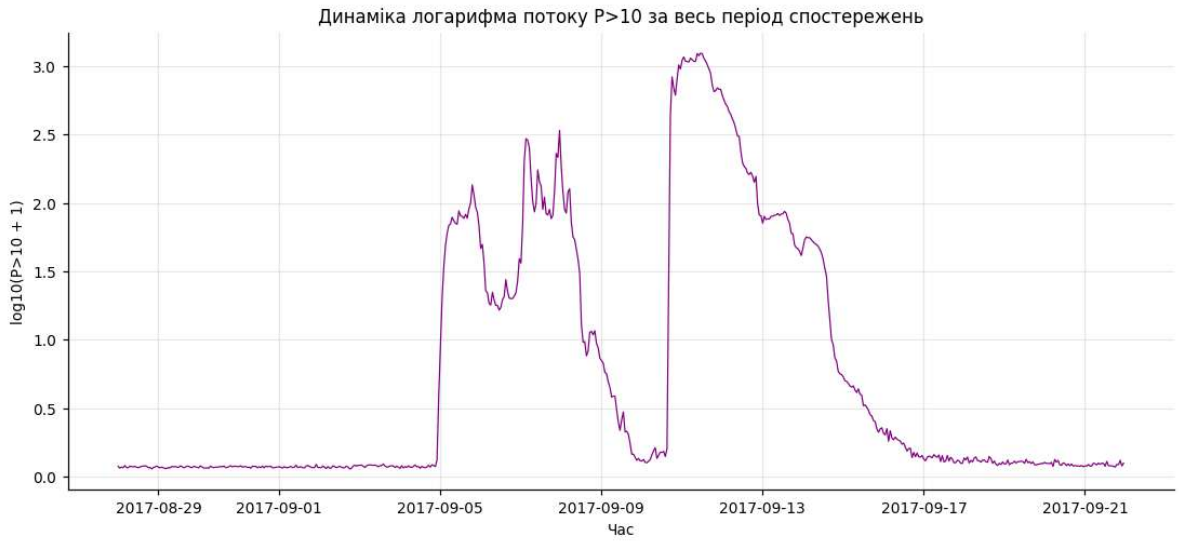
Якщо крива виявиться нечитабельною — згадайте, що допомогло в графіку 1.

```
# ВАШ КОД
import matplotlib.pyplot as plt
import numpy as np

plot_data = M.sort_values('ts')

plt.figure(figsize=(12, 5))
plt.plot(plot_data['ts'], np.log10(plot_data['P>10'] + 1), color='purple', l

plt.title('Динаміка логарифма потоку P>10 за весь період спостережень')
plt.xlabel('Час')
plt.ylabel('log10(P>10 + 1)')
plt.grid(True, alpha=0.3)
plt.show()
```



Висновок: На графіку чітко виділяються два потужні багатоденні періоди підвищеної сонячної активності (на початку та в середині вересня 2017 року), які характеризуються поступовим наростанням та плавним спаданням потоку протонів на фоні тривалих інтервалів спокою

Етап 6. Два графіки одних і тих самих даних

Нижче побудовано два графіки. Дані в них **одні й ті самі** — це той самий стовпець з тієї самої таблиці, жодне число не змінено.

Виконайте клітинку й подивіться на обидва.

```
СТОВПЕЦЬ = 'P>10'   # <- впишіть назву свого стовпця з МАКСИМУМОМ P>10 за го

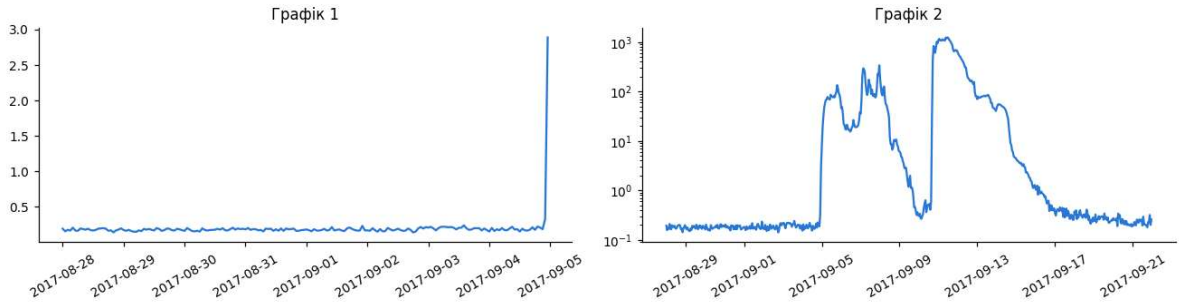
вікно = M[(M['ts'] >= '2017-08-28') & (M['ts'] < '2017-09-05')]

fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))

ax[0].plot(вікно['ts'], вікно[СТОВПЕЦЬ], color='#2a78d6')
ax[0].set_title('Графік 1')
ax[0].tick_params(axis='x', rotation=30)

ax[1].plot(M['ts'], M[СТОВПЕЦЬ], color='#2a78d6')
ax[1].set_yscale('log')
ax[1].set_title('Графік 2')
ax[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()
```



Завдання 6.1

Дайте відповідь на три питання:

1. Який висновок про космічну погоду у вересні 2017 року зробить людина, яка бачила **тільки перший** графік?
2. Який висновок зробить людина, яка бачила **тільки другий**?
3. Чим ці два графіки відрізняються між собою? Назвіть **дві** відмінності — вони обидві є в коді вище.

Жодне число в обох графіках не підроблене. Різниця виникла тільки через те, як їх побудували.

Відповідь 6.1:

1. Висновок з першого графіка: Космічна погода у цей період повністю спокійна та стабільна, а в кінці інтервалу спостерігається чимось схожий на аномалію різкий короткочасний всплеск
2. Висновок з другого графіка: Космічна погода є вкрай динамічною протягом серпня–вересня 2017 року чітко простежуються кілька потужних, тривалих хвиль сонячної активності з різкими підйомами та поступовим спаданням на тлі тривалих періодів спокою
3. Дві відмінності між ними:

- Перший графік охоплює лише вузький інтервал (до 5 вересня), тоді як другий демонструє загальну картину за весь період спостережень
- Перший графік побудовано в лінійному масштабі, через що деталі та масштаби реальних викидів згладжуються, тоді як другий використовує логарифмічний масштаб

Завдання 6.2

Уявіть, що вам потрібно **одним графіком** чесно показати, що відбувалося в цьому періоді. Побудуйте його самі й підпишіть заголовком, який формулює висновок.

Поясніть у клітинці нижче, чому ви обрали саме такий вигляд.

```
# ВАШ КОД
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 4.5))

plt.plot(M['ts'], M['P>10'], color='#1f77b4', linewidth=1)
plt.yscale('log')

plt.title('Чесна динаміка потоку P>10: чергування тривалого спокою та потужн
plt.xlabel('Час (годинна сітка)')
plt.ylabel('Потік P>10 (лог. шкала)')
plt.grid(True, which="both", ls="--", alpha=0.4)

plt.tight_layout()
plt.show()
```